Automated test generation from the RISC-V Sail Golden Model

Technical proposal in response to the Request for Quotation. Derived from the project's Master Technical Plan, which remains the authoritative engineering source and is referenced throughout as the appendix set.

PROPOSAL · DESIGN INTENT · NO DELIVERY DATE, COST OR UNVERIFIED RESULT IS STATED IN THIS DOCUMENT

---

1 · Executive Summary

We propose to build a configuration-aware test-generation framework driven by the upstream RISC-V Sail Golden Model, focused on the Privileged Architecture, and to integrate that capability into the Golden Model's own CI.

Four decisions shape the engineering strategy. Each is forced by evidence recorded in the Master Technical Plan rather than by preference.

DECISION	WHY
Configuration is an architecture, not a set of constants	One configuration object, loaded from the Golden Model's own JSON, derives every downstream artefact — model invocation, generator legality set, toolchain flags, simulator ISA string, test metadata and coverage scope. Configuration is presently re-derived independently in four places, which produces failures indistinguishable from model defects: the most expensive false signal a verification framework can emit
The model dependency is made upstreamable before anything is built on it	The framework requires model entry points that do not exist upstream. Goals 1 and Deliverable 5 are both gated on decomposing and merging those changes, and merge latency is outside our control — so this work starts in the first phase and runs continuously
Validation integrity is a deliverable, not a quality attribute	The framework must make it structurally impossible to report a passing suite that checks nothing. Every coverage and pass-rate claim depends on it, so it gates all reporting work
Coverage is a reporting capability, not a control loop	The RFQ requires per-ELF and suite coverage. It does not require coverage-guided generation, which is scoped as future work behind a defined interface rather than built now

THE COMPLETENESS POSITION

PRIMARY COMPLETENESS CLAIM

100% of the enumerated reachable denominators for each required configuration, with every unreachable target explicitly justified.

The framework measures four distinct quantities and reports them separately: **reachable architectural coverage**, **Sail branch coverage**, **fault sensitivity**, and **differential agreement with an independent simulator**. Section 5 explains why these are not interchangeable and why the first, not the second, carries the completeness claim.

We do not claim complete ISA coverage, and no target percentage is set for branch coverage. Exhaustive testing of a single `ADD` at RV64 spans 2^{128} operand pairs, so no finite percentage over the operand space is meaningful; "ISA conditions" have no enumerable denominator, the specification being prose; and model branch coverage can reach 100% while required architectural behaviour is absent from the model entirely.

WHAT IS BEING DELIVERED

Five deliverables, unchanged from the Master Technical Plan. **Two of them — CI integration and the required Golden Model changes — are satisfied only when merged upstream**, and therefore carry the programme's highest risk regardless of engineering effort applied. This is stated here rather than discovered later.

SCOPE, STATED UP FRONT

`REQUIRED` **M-MODE**

CSR access across the model-derived register set, trap entry and return, privilege transitions, PMP enforcement, PMA scenarios, interrupt delivery and delegation, trap-cause enumeration — both XLENs.

`COMMITTED` **S-MODE**

Sv32 and Sv39 address translation with negative controls. Sv32 unblocks the entire RV32 privileged path and is the highest-value single item in the privileged track.

`FUTURE IN RFQ` **DEPTH**

Sv48, Sv57, PTE content features, full fault taxonomy, pointer masking, and the remaining privileged extensions — delivered if schedule permits, otherwise deferred with recorded justification.

`OUT OF SCOPE` **HYPERVISOR**

Not implemented, not costed, not scheduled. The RFQ permits this. An orphaned upstream draft exists and must be evaluated before any from-scratch proposal, should it later be required.

Of the 28 named privileged extensions the Golden Model declares, **14 are committed for this RFQ and 14 are future within it**. Section 4 gives the full reconciliation, per extension and per XLEN, so that what we are not committing to is as visible as what we are.

2 · Understanding of the RFQ

2.1 WHAT IS BEING ASKED

The RFQ seeks a test-generation framework driven by the Sail Golden Model, evaluated primarily on coverage of the Privileged Architecture ISA extensions *as implemented by that model*. The qualification is load-bearing: the evaluation target is the model, which is enumerable, rather than the architecture specification, which is prose and is not.

The remaining criteria — maintainability, open-source integration, demonstrated community skill, cost and delivery — indicate the deliverable is expected to outlive the engagement and be maintained by the Golden Model community.

2.2 THE STRUCTURAL PROBLEM THAT DETERMINES THE DESIGN

> **The Privileged Architecture is machine state, not instructions.** Several privileged extensions define no instructions at all and are reachable only through specific CSR addresses. Trap behaviour, address translation, protection checking and interrupt delivery are reached by *establishing machine state* and then executing an ordinary instruction. This is why CSR access is the first M-mode capability in the delivery order, and why trap and privilege-transition infrastructure precedes everything that depends on it — not because they are easy, but because nothing else in the privileged track is complete without them.

> **Sail occupies three roles at once** — the source of the instruction set, the oracle for expected behaviour, and the coverage target. A single model change moves three numbers simultaneously. The framework therefore attributes every artefact to a model revision, and distinguishes evolution from regression rather than allowing them to be confused.

2.3 WHAT WE TAKE THE RFQ TO REQUIRE

REQUIREMENT	OUR READING
Use the current up-to-date Golden Model	Consumed from upstream and tracked; no private fork sustained as an end state. Required model changes are decomposed and submitted upstream
Generate valid RISC-V ELF files	Valid means it assembles, links and executes on a simulator the project did not write
Organise tests by ISA extension where practical	Suite selectable by extension and configuration from metadata alone
Support Golden Model configurations, particularly those in Golden Model CI	Every configuration in the committed set passes all pipeline stages, or is reported unsupported with the failing stage and an architectural reason
Focus on the privileged ISA	M-mode required; S-mode virtual memory highly desirable; Hypervisor out of scope. Section 4
Coverage for an individual ELF and an entire suite	Isolated per-ELF replay with a reconciliation invariant, plus configuration-, extension- and revision-attributed suite coverage

REQUIREMENT	OUR READING
Open-source licence compatible with the Golden Model	Attributions complete, items requiring legal review identified

2.4 FOUR CATEGORIES KEPT DISTINCT THROUGHOUT

CATEGORY	HOW IT APPEARS IN THIS DOCUMENT
What the RFQ requires	Stated as requirement, traced in Section 14
What the proposed framework will do	Future tense; the substance of Sections 4–9
What has already been demonstrated	Section 3.3 only, drawn from the Master Technical Plan's recorded evidence, with its own status column. **Prototype evidence informing production implementation — not a claim of completion**
What remains to be implemented	Everything in the phase plan, Section 11

3 · Technical Objectives and Success Criteria

3.1 GOALS, AS THE RFQ STATES THEM

#	GOAL	OWNING PHASES
1	Use the current up-to-date Sail RISC-V Golden Model	P2
2	Generate valid RISC-V ELF files	P4
3	Organise generated tests by ISA extension where practical	P4, P9
4	Support Golden Model configurations, particularly those used by Golden Model CI	P1, P10
5	Focus on the privileged ISA — M-mode required, S-mode virtual memory highly desirable, Hypervisor out of scope	P6, P7
6	Coverage for an individual ELF and for an entire generated suite	P8
7	Open-source licence compatible with the Golden Model	P11

3.2 SUCCESS CRITERIA — MEASURABLE, NOT JUDGEMENT CALLS

Thirteen success metrics are defined in the Master Technical Plan (Appendix G). Each is measurable by execution or by observable upstream state. The ones an evaluator is most likely to want:

#	METRIC	SUCCESS CONDITION
1	Clean-environment usability	Install → configure → generate → execute → coverage completes on a machine that has never built the project, with no file editing and no undocumented prerequisite
2	Configuration end-to-end support	Every configuration in the committed set passes all nine pipeline stages, **or** is reported unsupported with the failing stage and an architectural reason
3	Privileged capability	Every required M-mode capability and Sv32 + Sv39 pass on the Golden Model and an independent simulator, **with every negative control failing**
4	Validation integrity	100% of sampled tests fail under expectation mutation; 100% of controls fail; validated count reconciles with executed count
5	Per-ELF coverage correctness	Union of per-ELF span sets equals the suite total, asserted on every run
6	Coverage attribution	Emitting a figure without configuration, model revision and exclusion scope is **impossible**, not merely discouraged
7	Privileged coverage improvement	Measured increase over a recorded baseline, with the residual enumerated and classified
10	Maintainability	An engineer outside the project adds a backend and a privileged scenario from the developer documentation alone
12–13	Deliverables 5 and 4	Upstream repository state: **merged**. The only two metrics the project cannot satisfy by engineering effort alone

No target percentage is set for coverage, for the reasons in Section 5.4.

3.3 CURRENT BASELINE — WHAT PROTOTYPE WORK HAS ESTABLISHED

The following is recorded evidence from prototype work, carried forward from the Master Technical Plan. **It is not a claim of completion.** It is stated so the evaluator can distinguish what is known to be feasible from what is proposed.

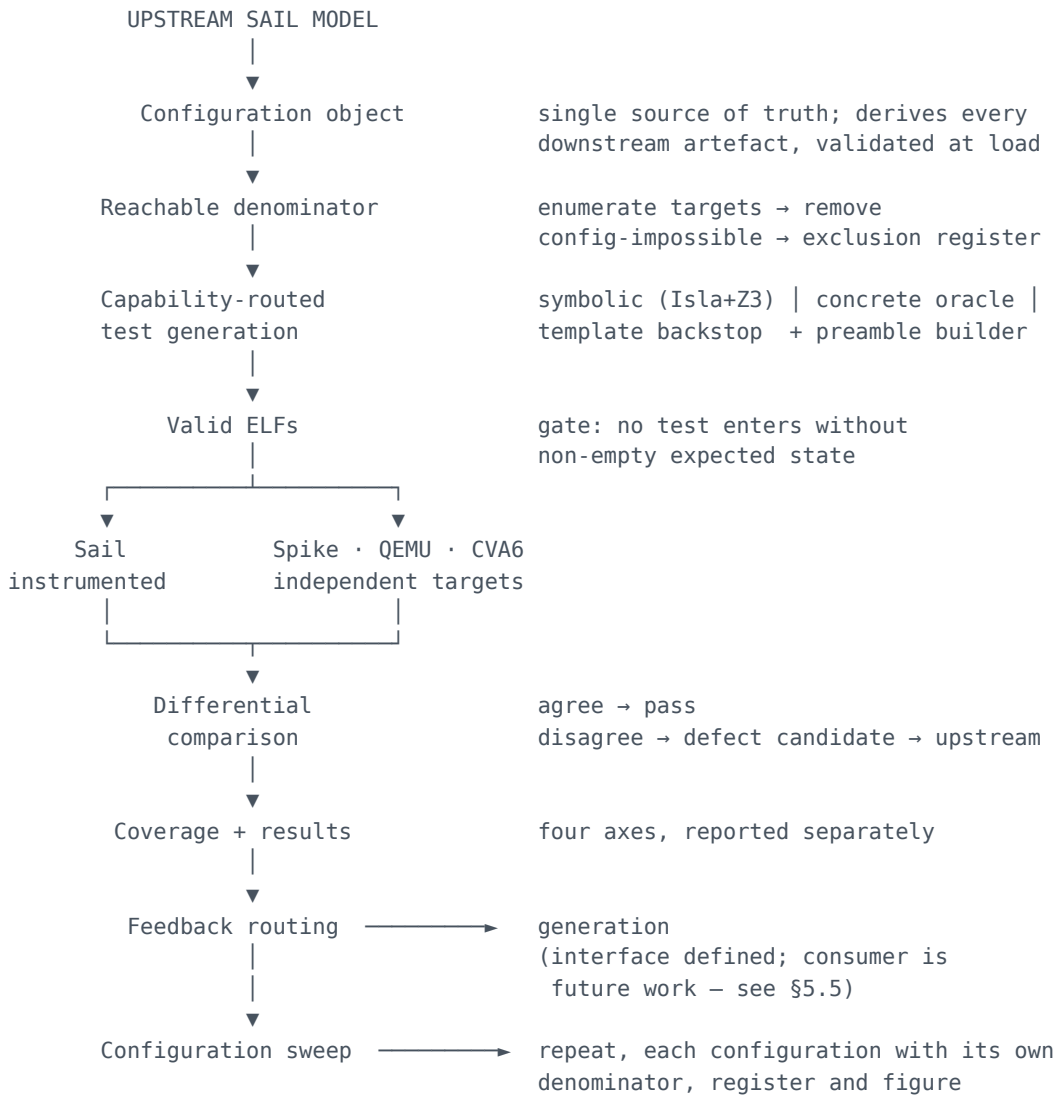
AREA	EVIDENCE	STATUS
ELF generation	Generated files are valid ELF32/ELF64, type EXEC, machine RISC-V, correct entry point and sections, and execute on an independent simulator	**Demonstrated**
Per-ELF and suite coverage	Isolated per-ELF replay with absolute and marginal contribution, and a reconciliation invariant asserting the suite union equals the union of per-ELF sets	**Demonstrated**
Privileged scenarios	Memory-protection violation, trap entry and return, privilege transitions, interrupt delivery and delegation, and 64-bit address translation — passing on the Golden Model *and* an independent simulator, with negative controls failing as required	**Demonstrated at one XLEN**
Backend routing	Floating point provably fails through the symbolic path and provably succeeds through the oracle	**Demonstrated**
Model-sourced instruction extraction	Instruction set derived from the model's own assembly declarations, with unparsed clauses reported rather than silently skipped	**Prototype**
Configuration consumption	The oracle consumes Golden Model configuration JSON directly, proving the format is usable unchanged	**Partial**
Circularity awareness	Tooling warns when only the Golden Model was run, because expectations derive from it	**Demonstrated**

3.4 KNOWN LIMITATIONS THE PLAN EXISTS TO RESOLVE

Twelve limitations are recorded in full in Appendix A. The four that shape the phase order: configuration is re-derived independently in four places; the framework currently requires a modified Golden Model; there is no mechanical anti-vacuity enforcement, and a prior corpus passed at scale while comparing empty expected-state tables; and address translation exists only for the 64-bit scheme, so RV32 translation scenarios *skip* — indistinguishable from success in a summary.

Two are structural boundaries rather than defects: the symbolic engine cannot represent vector lengths above its bitvector bound, and cannot execute floating-point arithmetic or vector element paths. These force the hybrid generation methodology of Section 6 and are documented as architecture, not treated as work items.

4 · Proposed Architecture



4.1 READING THE PIPELINE — WHAT EACH STAGE DOES AND WHY IT IS THERE

The ten stages are not a workflow diagram drawn after the fact. Each exists because omitting it produces a specific, identifiable failure, named in the third column.

#	STAGE	WHAT IT DOES	WHY IT IS THERE — WHAT ITS ABSENCE CAUSES	PRODUCES
01	**Read inputs**	Ingests the unmodified upstream Sail model and one Golden Model configuration JSON for this run. The configuration is loaded once into a validated object that rejects illegal combinations	Configuration re-derived independently in several places lets a test pass on Sail and fail on the independent simulator purely from framework disagreement — **a false signal indistinguishable from a model defect**	Validated configuration object; provenance record

#	STAGE	WHAT IT DOES	WHY IT IS THERE — WHAT ITS ABSENCE CAUSES	PRODUCES
02	**Compute the reachable denominator**	Enumerates every measurable target from the model's own declarations, evaluates each against the configuration predicates, and removes what cannot execute — *before* anything is generated	Without it a per-configuration 100% target is **unreachable by construction**, since configuration-gated code cannot execute. The framework then reports failure on every run and cannot distinguish a real gap from a structural impossibility	Reachable denominator; exclusion register with a reason per entry
03	**Generate, routed by capability**	Selects a backend per target from a declarative routing table: symbolic (Isla + Z3), concrete oracle, or template backstop. The preamble builder runs across all three, establishing machine state	No single mechanism reaches the whole ISA. The symbolic engine cannot execute floating-point arithmetic or vector element paths, and cannot represent vector lengths above its bitvector bound — **recorded structural boundaries, not defects**	Candidate tests: preamble + instruction sequence + expected state
04	**Gate: reject tests that cannot fail**	Checks every generated test before it enters the suite. Empty expected state is rejected; rejections are recorded with cause	A test with no expected state still executes and still raises the coverage figure, but cannot report a defect. **A prior corpus passed at scale while comparing empty expected-state tables**	Validated corpus; rejection record
05	**Execute on two targets**	Runs each ELF on the instrumented Golden Model and on an independently written simulator, under the same configuration	Generating from Sail, running on Sail and measuring against Sail is a **closed loop that cannot find a model defect** — every check passes on consistently wrong behaviour	Two execution records; executed branch spans
06	**Compare**	Differentially compares the two records. Agreement passes; divergence is triaged, minimised to a reproducer and submitted upstream. The Golden Model is never patched privately	Where the model *omits* required behaviour there is no branch to cover, so **no metric derived from the model can detect it**. Divergence is the only signal that surfaces it	Verdicts; model-defect candidates with evidence

#	STAGE	WHAT IT DOES	WHY IT IS THERE — WHAT ITS ABSENCE CAUSES	PRODUCES
07	**Measure**	Reachable architectural coverage (matrices), Sail branch coverage, fault sensitivity, and differential agreement — computed and reported separately	One number cannot carry this. Branch coverage measures the implementation and moves with how the model is written; only mutation establishes whether reaching a target verified anything	Matrix fill rates; coverage figures carrying configuration, revision and exclusion scope; mutation score
08	**Report**	Per-ELF records, suite summary, exclusion register and defect reports. Failures split into **model fault** and **framework fault**	A single pass/fail count is not actionable, because the two fault classes require entirely different responses	Machine-readable result file; generated human summary
09	**Classify and route feedback**	Classifies each uncovered target by cause — needs machine state, needs path enumeration, needs different operands, or unreachable here — rather than treating every gap identically	A single generic feedback path assumes every uncovered target is a generation-capability problem, when most privileged gaps are machine state the generator never established	Classified work queue. **Interface only in this RFQ — see below**
10	**Iterate across configurations**	Repeats the pipeline across the agreed matrix. Each configuration carries its own denominator, exclusion register and coverage figure	Averaging configurations with different denominators produces a quantity that is not meaningful and conceals where the gaps are	Per-configuration status matrix, generated by execution

Three points where the architecture diagram is broader than what this RFQ commits to. The figure describes the framework's design; Section 5 states the contractual scope, and where they differ Section 5 governs.

- **Stage 03, preamble builder.** The figure shows page-table construction for Bare, Sv32, Sv39, Sv48 and Sv57. **Sv32 and Sv39 are committed; Sv48 and Sv57 are future within this RFQ** — the same harness serves them, which is why they appear in the design, but they are delivered only if the schedule permits.
- **Stage 03, concrete oracle.** The figure shows the vector instruction set as a routing target. **Vector is deferred this iteration** — every vector entry in the Section 5.3 tables is marked future. The oracle is the correct route for it when it is in scope, and the routing table needs no change at that point.
- **Stage 09, feedback routing.** Shown closing the loop back to generation. **The RFQ requires per-ELF and suite coverage, not coverage-guided generation.** Stage 09's classification and its machine-readable uncovered list are delivered; the automated consumer that regenerates from them is scoped as future work behind that interface. This is the eighth extension point in Section 8.2.

4.2 ARCHITECTURAL PRINCIPLE

> **ISA-independent generation infrastructure is separated from privileged-specific scenario definitions.** Instruction sourcing, encoding, emission, execution, validation and coverage form a core that carries no privileged assumption. Privileged behaviour is expressed as scenario *data* consumed by that core, not as logic embedded in it. This is what makes later unprivileged-ISA generation a matter of adding inputs rather than restructuring the framework, and it is the foundation of the maintainability argument in Section 8.

4.3 CONFIGURATION AS THE SINGLE SOURCE OF TRUTH

One configuration object, loaded from the Golden Model's own JSON and validated at load so illegal combinations are rejected before anything is generated. From it derive: model invocation, generator legality set, toolchain flags, simulator ISA string, test metadata and coverage scope.

The alternative — the present state — is configuration re-derived independently in four places. A test can then pass on the Golden Model and fail on the independent simulator purely because the framework disagreed with itself, and that failure is indistinguishable from a model defect. Removing that class of false signal is why configuration architecture is the first phase.

4.4 COMMITTED CONFIGURATION SCOPE

Two questions must not be conflated: **generation support** means the framework can produce tests for a configuration; **end-to-end support** means generation, compilation, ELF, Golden Model execution, independent-simulator execution, validation and coverage all succeed. **Only end-to-end support is claimed as supported.**

CONFIGURATION	COMMITMENT	BASIS
RV32	END-TO-END COMMITTED	Generation, execution on two simulators and coverage demonstrated at this XLEN
RV64	END-TO-END COMMITTED	As above
F/D at RV64	END-TO-END COMMITTED	Generation and independent-simulator execution demonstrated via the oracle backend
F/D at RV32	TARGET, NOT COMMITTED	**Unknown — requires verification.** Resolved in P10; if it fails, recorded as unsupported with the failing stage named
Vector length / element width combinations in the model's own matrix	TARGET, WITH A STATED BOUND	Oracle path demonstrated at a non-default vector length. The symbolic path has a hard representational upper bound, so configurations above it are oracle-only by design
Combinations the Golden Model itself excludes	NOT COMMITTED	Excluded upstream for runtime reasons; included only if separately justified

The populated matrix is generated by execution in P10, not asserted here. Populating it in advance would be exactly the unsupported claim this plan forbids. An unsupported configuration will be reported as unsupported, not omitted.

5 · Privileged ISA Coverage Strategy

This is the RFQ's primary evaluation criterion and receives the most space in this proposal.

5.1 DELIVERY ORDER FOR M-MODE — REQUIRED

Ordered by technical dependency, not by visibility.

#	CAPABILITY	WHY IN THIS POSITION
1	**CSR access** across the model-derived register set, both XLENs	Several privileged extensions define no instructions at all and are reachable only through specific CSR addresses. Nothing else in M-mode is complete without it
2	**Trap entry, trap return, privilege transitions**	Infrastructure for everything below — subsequent scenarios need a trap handler and a way to change privilege
3	**PMP enforcement**	Required by the RFQ; observable only from a lower privilege mode, so it depends on 2
4	**PMA scenarios**, both XLENs	Region attributes are configuration-driven; requires the configuration architecture to vary them legitimately
5	**Interrupt delivery and delegation**	Requires trap infrastructure; delegation additionally requires supervisor entry
6	**Trap-cause enumeration**	Breadth pass once the mechanisms exist

> **An architectural constraint we document rather than solve.** Machine-level interrupt delegation is prevented by the model's own legalisation of the delegation register. Delegation scenarios therefore target supervisor-level interrupts. This is recorded so that it is not re-investigated by a future maintainer.

5.2 DELIVERY ORDER FOR S-MODE — HIGHLY DESIRABLE

#	CAPABILITY	COMMITMENT	RATIONALE
1	**Sv32**	COMMITTED	Unblocks the entire RV32 privileged path. Without it, RV32 translation scenarios *skip*, which is indistinguishable from success in a summary. **Highest-value single item in the privileged track**
2	**Sv39**	COMMITTED	RV64 baseline translation scheme
3	Sv48, Sv57	FUTURE IN RFQ	Depth variants against the same harness; low marginal cost once 1–2 exist
4	PTE content features — hardware A/D update, trap on improper A/D, NAPOT contiguity, page-based memory types, reserved-for-software bits	FUTURE IN RFQ	Content variants requiring a working walk harness

#	CAPABILITY	COMMITMENT	RATIONALE
5	Fault taxonomy — page faults, access faults, permission violations, U/S access rules	FUTURE IN RFQ	Breadth pass across 1–4

Every S-mode item depends on M-mode items 1 and 2. Items 1–2 appear in the Definition of Done and in success metric 3. **Items 3–5 are future within this RFQ: delivered if the schedule permits, otherwise explicitly deferred with recorded justification rather than silently omitted.** This distinction exists so that no stakeholder is led to expect Sv48/Sv57 and the full PTE feature set as contractual content.

5.3 COVERAGE BY EXTENSION — THE FULL RECONCILIATION

The Golden Model declares 28 named privileged extensions — 4 `Sm*`, 12 `Ss*` and 12 `Sv*`. With the three privilege-mode extensions `S`, `U` and `H`, the privileged portion of the model's extension enum totals 31 of 125 declared extensions. The table reconciles to all 28 named extensions so a reader sees the whole set rather than inferring the residual. **XLEN applicability is taken from the model, not from convention.**

✅ COMMITTED ○ FUTURE IN RFQ **—** not applicable, enforced by the model ✗ OUT OF SCOPE

> **WHY THE PRIVILEGED ARCHITECTURE IS REACHED THROUGH STATE, NOT OPCODES.** THIS MODEL IMPLEMENTS THE ENTIRE PRIVILEGED ARCHITECTURE THROUGH **NINE INSTRUCTIONS** — ECALL, EBREAK, MRET, SRET, WFI, SFENCE.VMA, AND SVINVAL'S SINVAL.VMA, SFENCE.W.INVAL AND SFENCE.INVAL.IR. COUNTING ONLY THOSE MEANINGFUL EXCLUSIVELY IN A PRIVILEGED CONTEXT GIVES SEVEN. ALMOST THE ENTIRE PRIVILEGED SURFACE IS THEREFORE REACHED THROUGH MACHINE STATE AND CSR ADDRESSES THE PREAMBLE CONSTRUCTS, NOT THROUGH OPCODE SELECTION. THAT IS WHY THE EXTENSIONS BELOW MAP OVERWHELMINGLY TO NO INSTRUCTION AT ALL, AND WHY THE PREAMBLE BUILDER RATHER THAN THE INSTRUCTION GENERATOR IS THE LOAD-BEARING COMPONENT OF THIS SECTION.

M-MODE — REQUIRED

EXTENSION / CAPABILITY	RV32	RV64	F/D	V	HOW IT IS REACHED
PMP *(base architecture)*	✅	✅	✅	○	Enforcement scenario + controls
PMA *(base)*	✅	✅	✅	○	Region-attribute configurations
Traps / exceptions *(base)*	✅	✅	✅	○	Expected-cause scenarios
Interrupts + delegation *(base)*	✅	✅	✅	○	Delivery and delegation scenarios
Privilege transitions *(base)*	✅	✅	✅	○	`mret` / `sret` into M, S, U
S (supervisor mode)	✅	✅	✅	○	Privilege transitions + S-mode entry
U (user mode)	✅	✅	✅	○	Privilege transitions
Zicsr	✅	✅	✅	○	Instruction sweep (6 mnemonics)
Zicntr	✅	✅	✅	○	CSR sweep — RV32 adds high-half registers
Zihpm	✅	✅	✅	○	CSR sweep
Stateen (Sm/Ss)	✅	✅	✅	○	CSR sweep — RV32 adds high-half registers
Smcntrpmf	✅	✅	✅	○	CSR sweep — RV32 adds high-half registers
Sscofpmf	✅	✅	✅	○	CSR sweep
Sscounterenw	✅	✅	✅	○	CSR sweep
Ssqosid	✅	✅	✅	○	CSR sweep
Sstc	✅	✅	✅	○	CSR sweep — RV32 adds the high-half register
Sstvala	○	○	○	○	CSR-field guarantee; needs trap-scenario inspection
Pointer masking — Smmpm, Smnpm, Ssnpm	○	○	○	○	**No new instructions.** Re-runs existing instruction tests under a masking configuration

S-MODE — HIGHLY DESIRABLE

EXTENSION	RV32	RV64	F/D	V	NOTES
Sv32	✅	—	✅	○	RV32-only by model gating. **Unblocks the entire RV32 privileged path**
Sv39	—	✅	✅	○	RV64 baseline translation scheme
Sv48	—	○	○	○	Depth variant on the same harness
Sv57	—	○	○	○	Depth variant on the same harness
Svbare	○	○	○	○	No-translation mode — implicitly the state of every test today, never deliberately verified
Svinval	✅	✅	✅	○	Has 3 instructions; covered by the instruction sweep
Svade	○	○	○	○	PTE content: trap on improper A/D
Svadu	○	○	○	○	PTE content: hardware A/D update
Svnapot	—	○	○	○	PTE content: NAPOT contiguity. RV64-only by model gating
Svpbmt	—	○	○	○	PTE content: page-based memory types. RV64-only by model gating
Svrsw60t59b	○	○	○	○	PTE content: reserved-for-software bits
Svvptc	○	○	○	○	Obviating re-fence after PTE update
Ssccptr	○	○	○	○	Hardware PTE-read guarantee; a memory property, not an opcode set

RECONCILIATION

Committed this RFQ — **14**
Future within this RFQ — **14**
Total named privileged extensions — 28

THE CONFIGURATION MATRIX, ENUMERATED

“Per configuration” appears throughout this document. These are the configurations meant, taken from the Golden Model's own test suite rather than invented here. The model ties XLEN to ELEN and skips VLEN 512 with ELEN 32, which gives **seven** configurations rather than the sixteen a free cross-product would suggest.

#	CONFIGURATION	XLEN	VLEN	ELEN	SYMBOLIC PATH
1	`rv32d_v64_e32`	32	64	32	Available
2	`rv64d_v64_e64`	64	64	64	Available
3	`rv32d_v128_e32`	32	128	32	Available
4	`rv64d_v128_e64`	64	128	64	Available
5	`rv32d_v256_e32`	32	256	32	Oracle only — VLEN exceeds the engine's 129-bit bound

#	CONFIGURATION	XLEN	VLEN	ELEN	SYMBOLIC PATH
6	`rv64d_v256_e64`	64	256	64	Oracle only
7	`rv64d_v512_e64`	64	512	64	Oracle only

All seven are committed: each gets its own reachable target list, its own exclusion list and its own coverage figures, never averaged together. The right-hand column is a routing consequence rather than a coverage gap — the concrete oracle runs the model directly and has no width bound, so configurations 5 to 7 are generated and measured like any other.

On the F/D and V columns

Privileged behaviour is largely orthogonal to floating point, but not entirely: `mstatus.FS` affects illegal-instruction trapping, so privileged scenarios run under F/D-enabled configurations as part of the configuration matrix rather than as separate work. **Every V entry is ○ because vector is deferred this iteration** — privileged scenarios are not blocked by vector, but privileged × vector combinations are not committed.

Hypervisor is out of scope for this iteration. The RFQ permits this. Not implemented, not costed, not scheduled. If later required, an orphaned upstream draft implementation exists and must be evaluated before any from-scratch proposal.

5.4 WHY COVERAGE NEEDS A DENOMINATOR BEFORE GENERATION BEGINS

A coverage figure is a proportion, and a proportion requires a denominator. The denominator is not a constant: a substantial part of the model cannot execute in a given configuration, by construction. The model gates `Sv32` on `xlen == 32`, and `Sv39`, `Sv48`, `Sv57`, `Svnapot` and `Svpbmt` on `xlen == 64`; a large number of further sites are guarded on XLEN directly.

> **Without this step a per-configuration 100% target is unreachable by construction, because configuration-gated code cannot execute.** A framework that adopts 100% as a goal without first computing reachability reports failure on every run, and cannot distinguish a genuine gap from a structural impossibility.

Procedure, before generation: enumerate all measurable targets from the model's own declarations; evaluate each target's reachability under the configuration predicates; remove what cannot execute; emit an **exclusion register** with one entry per excluded target and a reason from a closed vocabulary — `config-gated`, `extension-absent`, `not-implemented`, `platform-description`, `out-of-scope`.

Why each exclusion needs a reason. The register is reviewable line by line. A reader who disagrees contests one entry, on its stated grounds, rather than disputing an aggregate figure with no visible basis. Free-text justification is not permitted, which is why the vocabulary is closed.

5.5 FOUR MEASUREMENT AXES, REPORTED SEPARATELY

AXIS	WHAT IT ANSWERS	CONSTRUCTION AND STATUS
A · Reachable architectural coverage	Did we reach every enumerated target that this configuration can execute?	**Completable — this axis carries the completeness claim.** Trap matrix (exception causes × delegated/not × originating privilege); CSR access matrix across the model-derived register set; translation matrix (mode × page size × permissions × A/D × PMP interaction); operand boundary partitions. Fill rate reported per configuration against the reachable denominator
B · Sail branch coverage	What model behaviour has the suite not reached?	**Measured and reported; never the objective.** Per-ELF isolated replay with the reconciliation invariant, plus suite coverage attributed by configuration, extension and model revision. Every figure carries configuration, model revision and exclusion scope — emitting one without them is impossible by construction
C · Fault sensitivity	Would the suite notice if the model were wrong?	**The audit on A and B.** Sampled expectation mutation; 100% of sampled tests must fail. Full matrix fill combined with poor fault sensitivity indicates the partitions are wrong, not that the work is complete
D · Differential agreement	Does an independently written implementation agree?	**The only axis that can detect absent model behaviour.** Agreement rate with an independent simulator, every divergence classified. Section 7

5.6 WHY THE MATRICES GIVE A MORE DEFENSIBLE CLAIM THAN "BRANCH COVERAGE = X%"

Three reasons, each independent.

- **The matrices are completable; branch coverage is not.** A trap matrix cell is either filled by a test producing exactly that cause with the correct trap value and resulting privilege, or it is registered as unreachable with a reason. There is no third state. A branch-coverage percentage has no such structure — it is a ratio whose numerator and denominator both move when the model changes.
- **Branch coverage measures the implementation, not the architecture.** Whether an architectural corner case is visible to it depends on how the model author wrote the line: an operation implemented as a single expression has no branches, so one trivial test achieves complete branch coverage of it while leaving overflow and boundary behaviour untested. A matrix cell is defined by architectural meaning, so it does not move with implementation style.
- **Branch coverage cannot see absent behaviour.** Where the model omits required behaviour there is no branch to cover, and a complete figure is reported on a defective model. Axis D exists for precisely this case.

5.7 HOW PRIVILEGED PROGRESS IS REPORTED

As a structure, never as one number: a measured baseline at a stated model revision and configuration before privileged work begins; per-extension coverage so a gap attributes to a specific extension; matrix fill rates per configuration; marginal contribution per capability delivered; negative-control status for every scenario family, where a control that passes is reported as a framework defect; an explicit uncovered list with each entry classified reachable, unreachable here, or out of scope; and trend across model revisions so evolution is distinguishable from progress.

> **We do not claim complete ISA coverage.** Exhaustive testing of a single `ADD` at RV64 spans 2^{128} operand pairs — for one instruction, before machine state is considered — so no finite percentage over the operand space is meaningful. Equivalence-class partitioning is the necessary response, and we publish the partition so its adequacy can be argued with rather than assumed. "ISA conditions" provide no enumerable denominator, the specification being prose. Every figure this framework publishes is a proportion of a denominator it states.

6 · Test Generation Strategy

No single mechanism solves the whole ISA, and the plan does not pretend otherwise. Backends are selected by a **declarative routing table** rather than by hard-coded dispatch, which is also what makes a backend replaceable (Section 8).

SYMBOLIC · ISLA + Z3

Isla executes the Sail model symbolically: instead of concrete register values it carries unknowns and accumulates the constraints a chosen execution path implies. Handing those to the Z3 solver yields concrete operands that drive the model down that path — which is what is wanted when the target is a specific model behaviour.

Requires model entry points that do not exist upstream. See Section 9 and the fallback position.

CONCRETE ORACLE

Selects values and runs the model to obtain the resulting architectural state, which becomes the test's expected state. Builds no constraint system, so solver-level representational limits do not apply.

Consumes Golden Model configuration JSON directly — **demonstrated**, which is what establishes that the format is usable unchanged.

> **Routing is forced by two recorded structural boundaries, not by preference.** The symbolic engine cannot represent vector lengths above its bitvector bound, and cannot execute floating-point arithmetic or vector element paths. Floating point provably fails through the symbolic path and provably succeeds through the oracle — this is recorded evidence, and it is why the methodology is hybrid. Configurations above the vector bound are **oracle-only by design**, stated as an architectural boundary rather than as a defect to be fixed.

6.1 TEMPLATE BACKSTOP

A small, capped set of builders for constructs neither engine reaches cleanly. **Growth in this tier is treated as a routing defect signal rather than as progress**, and the count is reported in every suite summary for that reason.

6.2 PRIVILEGED PREAMBLE CONSTRUCTION

Because privileged behaviour is reached by machine state, the component that establishes machine state determines privileged coverage. The preamble builder constructs privilege level, status-register fields, page tables for the committed translation schemes, PMP regions, PMA configuration, delegation configuration and pending-interrupt state.

Machine state is written into the preamble rather than asserted as a solver constraint. Two consequences: state setup stays independent of which backend produces the instruction body, so all routes benefit; and constructing a page table directly is more predictable than asking a constraint solver to discover one. Where the symbolic backend is unavailable — the fallback position of Section 9 — privileged scenarios relying on solved state move to preamble construction, which is a capability reduction rather than a failure.

6.3 ELF EMISSION AND PORTABILITY

One assembly format across all backends, with annotation that leaves the encoding byte-identical. Implementation-dependent operations — boot, termination, interrupt conventions — are isolated in their own layer, which is what makes generated tests portable to a simulator the project did not write and what supports Architectural Certification Test compatibility.

6.4 THE QUALITY GATE

> **The framework must make it structurally impossible to ship a test that cannot fail.** A prior corpus passed at scale while comparing empty expected-state tables. Mechanical anti-vacuity enforcement is therefore a deliverable in its own right: no test enters the suite without a non-empty expected state, rejections are recorded with cause, and pass rates are computed only over validated tests. **Introducing this is expected to reduce apparent pass rates, and that is disclosed in advance rather than presented as a regression.**

7 · Validation and Differential Execution

7.1 WHY SAIL-ONLY EXECUTION IS INSUFFICIENT

> **Generating from Sail, running on Sail and measuring against Sail is a closed loop that cannot find a model defect.** If expected state derives from the model, executes on the model, and is scored against the model's own coverage instrumentation, then any behaviour the model implements incorrectly is reproduced consistently at every stage and every check passes. The framework already warns when only the Golden Model has been run, for exactly this reason.

The subtler case is absent behaviour. Where the model omits something the architecture requires, there is no branch to cover, so no metric computed from the model can point at it. Divergence against an independently written implementation is the only mechanism in this design capable of surfacing it.

7.2 THE TWO TARGETS

SAIL — INSTRUMENTED

Reference behaviour and the executed span record. Coverage collection is decoupled from reporting so an alternative coverage source can be added without redesign.

INDEPENDENT SIMULATOR

The same ELF under the same configuration, executed by an independently implemented model. Required by Deliverable 3, which specifies a simulator the project did not write.

Execution backends sit behind an interface, so additional simulators are an extension rather than a rewrite.

7.3 COMPARISON AND OUTCOMES

OUTCOME	ACTION
Agree	Test passes. Coverage recorded against the reachable denominator for that configuration
Disagree	Model-defect candidate. Triaged, minimised to a reproducer, and submitted upstream. **The Golden Model is not patched privately** — a locally patched model is no longer the Golden Model, and coverage measured against it is not comparable
Only Sail available	**Reported inconclusive, never pass.** This is an explicit acceptance condition of Deliverable 3

Divergence is not automatically a model defect. Triage distinguishes a Sail defect, an independent-simulator defect, a framework fault — most commonly configuration disagreement, which the single-source configuration object of Section 4.2 is designed to eliminate — and a legitimate implementation-defined difference. The classification is recorded per divergence with its evidence.

7.4 VALIDATION INTEGRITY AS A GATING DELIVERABLE

Failures are classified into **model fault** and **framework fault** and reported separately; a single pass/fail count without that split is not actionable, since the two require entirely different responses.

Negative controls accompany every privileged scenario family, and a control that passes is reported as a framework defect rather than absorbed.

> **One structural problem, stated rather than hidden.** The model's coverage record is written only on clean exit, so failing tests contribute no coverage. Because negative controls must fail, the very tests that prove the suite is non-vacuous are structurally invisible to coverage. Coverage accounting therefore separates the validated corpus from the control corpus, and every figure is published with its scope and with the statement that it reflects passing tests only.

8 · Maintainability and Sail Evolution

Directly addressing RFQ criterion 2. The framework is expected to be maintained by the Golden Model community after delivery.

8.1 THE RESPONSE WHEN UPSTREAM SAIL CHANGES

```
New Sail functionality
        ↓
Model analysis                  instruction set and declarations re-read from the model
        ↓
New targets become visible      without a maintainer editing any list
        ↓
Reachable denominator updated   per configuration; exclusion register regenerated
        ↓
Existing generation             generic backends attempt the new targets first
capabilities attempt coverage
        ↓
New capability adapter          only where a genuinely new architectural mechanism
only where genuinely required   requires special initialisation
```

The maintenance burden in a framework of this kind is not the generation engines — it is keeping the framework's understanding of the model synchronised with a model under active development. Hand-maintained lists of instructions, CSRs and extensions are the artefacts that rot. **Instruction sourcing is therefore model-driven: the instruction set is derived from the model's own assembly declarations, with unparsed clauses reported rather than silently skipped**, so a new extension appears without framework change.

8.2 THE EIGHT EXTENSION POINTS

EXTENSION POINT	WHAT CAN BE ADDED LATER	WHY IT IS AN INTERFACE NOW
Instruction source	New ISA extensions	Instruction set derived from the model, so a new extension appears without framework change
Generation backend	New generation strategies	Backends selected by a declarative routing table
Scenario definition	New privileged scenarios; Hypervisor scenarios	Scenarios are declarative state descriptors, not fixed encodings
Machine-state descriptor	Floating-point state, vector state, Hypervisor state	State requirements are configuration-derived fields
Implementation-dependent operations	Alternative boot / termination / interrupt conventions	Isolates portability from generation
Execution backend	Additional simulators	Required anyway by Deliverable 3
Validation mechanism	Trace-based or other pass/fail schemes	The RFQ permits schemes other than self-checking

EXTENSION POINT	WHAT CAN BE ADDED LATER	WHY IT IS AN INTERFACE NOW
Coverage source	Alternative coverage mechanisms	Reporting decoupled from collection

> **Scope discipline.** These are interfaces, not implementations. No Hypervisor, floating-point, vector or additional-simulator capability is built in this project beyond what the RFQ requires. The distinction is deliberate: an interface that is defined but unexercised is honest; an implementation claimed but not built is not.

8.3 MODEL EVOLUTION HANDLED EXPLICITLY

Sail's triple role means one model change moves three numbers at once. Every artefact therefore carries model-revision attribution, coverage figures are revision-qualified, and differences between revisions are classified as evolution or regression rather than left ambiguous. Regeneration against a newer model revision is exercised as an acceptance activity, not assumed to work.

8.4 MAINTAINABILITY PROVEN, NOT ASSERTED

Two acceptance conditions test this directly rather than taking it on trust: **an engineer outside the project adds a backend and a privileged scenario using the developer documentation alone**, and a clean machine completes install → configure → generate → execute → coverage from documentation alone, with no file editing and no undocumented prerequisite. The framework's own test suite runs green in CI and gates every change, with a deliberately introduced regression demonstrating that it catches one.

9 · Open Source Integration

Directly addressing RFQ criterion 3. **Invoking a project is not integrating with it**, so relationships are stated at the level the actual interaction supports.

PROJECT	RELATIONSHIP	BASIS
`riscv/sail-riscv`	**UPSTREAMING**	Required framework changes and any model defects found are submitted as pull requests; CI integration is a merged deliverable
`riscv-non-isa/riscv-arch-test`	**INTEGRATION**, contribution assessed	Generated tests made portable through the implementation-dependent operations layer; contribution decided by evaluation, not assumed
Isla	**CONTRIBUTION assessed**	The project extends the symbolic generator; extensions of general value are assessed for upstream contribution, with a recorded decision either way
Spike	**USE**	Independent execution reference. No contribution anticipated — **a relationship classified USE is stated as USE**
Golden Model community workflows	**INTEGRATION**	The framework fits existing conventions rather than introducing parallel ones

Each relationship carries an owner and a recorded outcome. **No community acceptance, merged contribution or endorsement is claimed in this proposal.** Deliverables 4 and 5 are defined by upstream merge state precisely so that the claim, when it can be made, is verifiable by inspection of a public repository rather than by assertion.

9.1 THE UPSTREAM DEPENDENCY, STATED CONCRETELY

This is the question a Golden Model reviewer asks first, so it is answered here rather than deferred. **The framework's symbolic generation path currently cannot run against unmodified upstream:** the two entry-point functions it requires are absent, and the IR build step requires them by name.

CHANGE	INVASIVENESS	UPSTREAMABLE	FALLBACK IF REJECTED
Symbolic entry points — an initialisation and a step entry point for external symbolic tooling	**Low** — additive, no change to existing behaviour	Likely	Symbolic backend unavailable; the framework operates oracle-only. Privileged scenarios relying on solved state move to preamble construction. **Capability reduction, not project failure**
CSR accessor restructuring — consolidation of scattered accessor definitions	**High** — large, touches a widely used area	**Unknown — requires verification.** The most likely change to be rejected	CSR-driven symbolic generation unavailable; CSR coverage moves entirely to the oracle path. **P2 must first test whether this change is eliminable**

CHANGE	INVASIVENESS	UPSTREAMABLE	FALLBACK IF REJECTED
Reset-behaviour fix — correction in memory-protection register reset	**Low** — narrow	Likely; carries its own justification	Not required by the framework; submitted as a standalone contribution regardless
Remaining files in the change set	Unknown	Unknown	Per-file triage is the first task of P2

> **The project does not assume any of these will be accepted.** P2's first task is to determine which are eliminable and remove them; the remainder are decomposed into single-purpose reviewable pull requests ordered least-contentious first. The present change set is a single large multi-file commit and is **not reviewable upstream as submitted** — decomposing it is scheduled work, not an afterthought. The high-invasiveness change is the one genuine threat to the symbolic backend, and is stated as such rather than presented as a formality.

9.2 TECHNICAL COMPETENCIES REQUIRED TO EXECUTE

Stated as the capability set the plan demands, so an evaluator can assess fit against RFQ criterion 4. **Evidence of these competencies in the delivering team is supplied separately with verifiable references; none is asserted here.**

DOMAIN	WHERE THE PLAN DEMANDS IT
RISC-V privileged architecture	Scenario design, trap and translation semantics, delegation, PMP/PMA — Section 5
Sail language and the Golden Model	Model-sourced instruction extraction, coverage instrumentation, upstream change decomposition — Sections 6, 9
Symbolic execution and SMT	Isla and Z3 integration, representational bounds, backend routing — Section 6
ELF generation and toolchain	Assembly, linking, portability layer, certification-test conventions — Section 6.3
Differential testing	Independent-simulator integration, divergence triage — Section 7
Functional coverage methodology	Denominators, matrices, mutation, attribution — Section 5
Python automation and verification infrastructure	Orchestration, reporting, CI — Sections 10, 11
Open-source contribution process	PR decomposition, upstream review engagement — Section 9.1

10 · Deliverables and Acceptance Criteria

Five deliverables, unchanged from the Master Technical Plan. Acceptance is by demonstration, not by assertion.

#	DELIVERABLE	WHY IT MATTERS	ACCEPTANCE EVIDENCE
1	**Repository with framework, user documentation and developer documentation**	The framework must be maintainable by the community after delivery — RFQ criterion 2	A clean machine completes clone → documented install → build → configure → generate → execute → coverage with no file editing and no undocumented prerequisite. Framework test suite green in CI. Developer documentation passes its acceptance test: **an outside maintainer adds a backend and a privileged scenario from the documentation alone**. Licence, attributions and notices complete, with items requiring legal review identified
2	**Example scripts generating a suite from a given configuration**	Demonstrates the configuration architecture works in practice, not only in design	A single documented command generates an extension-organised, manifested, provenance-stamped suite from **any** valid Golden Model configuration. Provenance sufficient to regenerate byte-identically. **Verified across the configuration matrix, not on one example**
3	**Example scripts executing a suite, collecting results, summarising**	Closes the loop from generation to a verdict a human can read	A single documented command executes a suite on a named simulator and produces per-test verdicts with runtime and failure classification, a machine-readable result file, and a human-readable summary. **Must operate on an independent simulator, and must report inconclusive rather than pass when only the Golden Model is available**
4	**Golden Model CI integration**	Makes the capability part of the Golden Model's own workflow rather than an external artefact	States tracked distinctly: local implementation → PR created → reviewed → approved → **merged**. **Only merged satisfies this deliverable**
5	**Required Golden Model changes**	Removes the private-fork dependency; Goal 1 depends on it	Per change: not required / identified / implemented locally / PR created / reviewed / approved / **merged**. **Only merged satisfies this deliverable.** Each required change carries a documented fallback for non-acceptance

> **For Deliverables 4 and 5, PR created ≠ complete; reviewed ≠ complete; approved ≠ complete; merged = complete.** These states are tracked separately from engineering progress throughout, so that programme status is never reported as complete on the strength of work that has been submitted but not accepted.

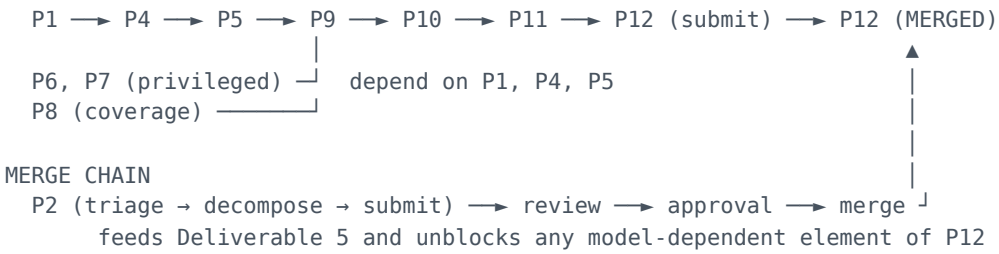
11 · Timeline and Milestones

Twelve phases across four tracks, totalling **15 engineering-weeks** from kickoff — one number rather than a range, so it can be held to. Per-phase durations below are stated as effort-bearing sequence and dependency, not as calendar dates. The figure assumes human-plus-AI-augmented delivery, realistic where a phase is well-scoped and mechanical, and most of this roadmap is.

Upstream review does not compress the same way engineering effort does; that latency is tracked as risk, not folded into the 15 weeks.

PH	OBJECTIVE	KEY DELIVERABLE CONTRIBUTION	ACCEPTANCE EVIDENCE
P1	Configuration architecture	Foundation for D1–D3	A single validated configuration object derives every downstream artefact; illegal combinations rejected at load; single-point derivation proven
P2	Model integration and upstream decomposition	**D5**	Per-file triage complete; eliminable changes removed; remainder decomposed into single-purpose reviewable PRs, submitted; ecosystem map recorded
P3	Framework productization and self-verification	D1	Installable in a clean environment; framework test suite green and gating; dependency manifest present
P4	Generation core and backend routing	D1, D2	Configuration-aware core with declarative routing; ELF validity verified by execution on an independent simulator; one assembly format across backends
P5	Validation integrity	D3	Shipping a test that cannot fail is structurally impossible; mutation sampling and control checks pass; validated count reconciles with executed count
P6	**M-mode privileged capability**	**Criterion 1**	Every required M-mode capability passes on two simulators at both XLENs, with every negative control failing
P7	**S-mode address translation**	**Criterion 1**	Sv32 and Sv39 scenarios pass on two simulators with controls failing; RV32 translation no longer skips
P8	Coverage architecture	D1, criterion 1	Per-ELF isolated replay with reconciliation invariant; configuration-, extension- and revision-attributed suite coverage; per-extension privileged view
P9	Suite, provenance and result architecture	D3	Reproducible self-describing suites; unified result record; generated human summary
P10	Configuration matrix validation	D2, criterion 1	Per-stage matrix generated by execution; unsupported configurations reported with the failing stage and an architectural reason
P11	Turn-key packaging, examples, documentation	**D1, D2, D3**	One command per configuration; both documentation tracks pass their acceptance tests; licensing complete
P12	Golden Model CI integration	**D4**	**Merged** pull request

11.1 CRITICAL PATH AND DEPENDENCIES



> **The binding constraint is the merge chain, not the engineering chain.** Its duration is set by third-party review and cannot be compressed by engineering effort. Two consequences for execution: the upstream track starts in the first phase and runs continuously rather than being scheduled at the end, and every required model change carries a documented fallback so that rejection degrades capability rather than stopping delivery.

Highest-risk critical-path items: P5, which may invalidate existing measurements and force rework in P6–P8; P10, which may reveal unsupportable configurations; and P12, whose completion is outside our control.

12 · Cost and Commercial Assumptions

> **No cost figure is stated in this proposal.** The Master Technical Plan deliberately assigns no costs and no dates, and inventing them here would misrepresent the basis on which they can be derived. What follows is the model by which a fixed price will be produced.

12.1 EFFORT MODEL

```
Total = Σ (phase effort × blended day rate)
      + configuration-matrix validation effort        scales with the agreed matrix
      + upstream engagement allowance                 review response, PR decomposition
      + contingency                                   against §13 risks
```

The **upstream engagement allowance is separated deliberately**: its duration is set by third-party review latency, and folding it into engineering effort would make the estimate look more predictable than it is.

12.2 COST DRIVERS, IN ORDER OF INFLUENCE

DRIVER	EFFECT
Size of the agreed configuration matrix	P10 effort and per-configuration validation scale directly with it
How many "future within RFQ" privileged items become committed	The 14 currently-future extensions are the largest discretionary block in the plan

DRIVER	EFFECT
Extent of required Golden Model change	P2 triage may eliminate changes, reducing effort; the high-invasiveness change may require substantial redesign of CSR handling if rejected
Upstream review latency	Affects elapsed time and engagement effort, not engineering effort. Outside our control
Choice of independent simulator(s)	Two simulators are unverified for execution and are scoped as verification tasks; a negative result adds integration work

12.3 WHAT IS REQUIRED TO FINALISE A FIXED PRICE

1. The agreed configuration matrix — which configurations must be end-to-end supported.
2. Which of the 14 "future within RFQ" privileged items are contractual rather than best-effort.
3. The agreed independent simulator, and whether more than one is required.
4. Whether Deliverables 4 and 5 are accepted as *merged* or as *submitted in good faith* — this materially changes risk exposure and therefore contingency.
5. Agreed start date, staffing profile and concurrency.
6. Whether framework maintenance beyond delivery is in scope.

> **Delivery date.** Derivable from the phase structure and critical path once a start date and staffing profile are agreed. **It cannot be fixed independently of items 1, 2 and 4 above**, and — for Deliverables 4 and 5 — depends on upstream review latency that no supplier can commit to on a third party's behalf. We would rather state this plainly than quote a date we cannot control.

13 · Risks and Dependencies

RISK	IMPACT	MITIGATION	RESIDUAL DEPENDENCY
Upstream rejects structural model changes	Strands the symbolic backend	Eliminate what can be eliminated in P2; decompose so rejection is isolated to one PR; documented fallback per change	**Maintainer decision.** Residual: oracle-only operation, a capability reduction
Merge latency for Deliverables 4 and 5	Cannot be compressed by effort	Upstream track starts in P1 and runs continuously; states tracked separately from engineering progress	**Review capacity.** Sets the programme's elapsed duration
Golden Model CI integration not accepted	Deliverable 4 unmet	CI consumes pre-generated pinned suites so runtime cost to upstream is bounded; generation runs on a schedule, not per commit	**Maintainer decision** on CI policy
Generation capability limits — vector length bound, no symbolic FP	Configuration matrix cannot be fully served symbolically	Documented architectural boundary; oracle routing above the bound; recorded rather than re-investigated	None — accepted as architecture

RISK	IMPACT	MITIGATION	RESIDUAL DEPENDENCY
Privileged machine-state complexity	P6–P7 effort exceeds estimate	Capabilities ordered by dependency so partial delivery degrades scope rather than blocking; scenario families independently deliverable	Scope reduction visible in the committed/future split
Configuration-specific reachability	Goal 4 partially unmet	Per-stage matrix with a documented architectural reason per failing stage, rather than a single supported/unsupported flag	Reported, not hidden
Vacuity enforcement invalidates existing numbers	Apparent large regression	**Expected and disclosed in advance**; pass rates computed only over validated tests from that point	None — a reporting correction
Sail's triple role — source, oracle, coverage target	One model change moves three numbers at once	Model-revision attribution on every artefact; evolution-versus-regression classification	Model release cadence
Carried-forward unknowns resolve negatively	Capability narrower than hoped	Each scoped as an explicit verification task in P6, P9 or P10; result recorded either way	Four open items, listed in Appendix A

The full engineering risk register is Appendix E.

14 · RFQ Evaluation Criteria Traceability

#	CRITERION	WHERE ADDRESSED	ACCEPTANCE EVIDENCE
1	**Coverage of privileged ISA extensions implemented by Sail**	§5 in full — delivery order (5.1–5.2), per-extension reconciliation (5.3), reachable denominator (5.4), four measurement axes (5.5), why matrices beat a branch percentage (5.6), progress reporting (5.7). Phases P6, P7, P8	Scenarios pass on two simulators with every negative control failing; per-extension privileged coverage against a recorded baseline; 14 committed extensions reconciled against 28
2	**Long-term maintainability and extensibility**	§8 in full — model-driven instruction sourcing, eight extension points, ISA-independent core separated from privileged scenario data, model-revision attribution. §4.2 single-source configuration. Phases P3, P11	**An outside maintainer adds a backend and a privileged scenario from documentation alone.** Clean-machine run with no file editing. Framework suite green in CI, catching a deliberately introduced regression
3	**Integration with open source communities**	§9 — USE / INTEGRATION / CONTRIBUTION / UPSTREAMING classification per project, with the upstream dependency stated concretely in 9.1. Phases P2, P12	**Merged** pull requests for Deliverables 4 and 5 — verifiable by inspecting a public repository. Ecosystem map with a recorded outcome per relationship
4	**Demonstrated technical skill in the targeted ecosystems**	§9.2 competency map; §3.3 recorded prototype evidence; §6 routing decisions justified from tool properties. **No artificial tasks are created to manufacture evidence**	Emerges from merged PRs, CI integration and reproducible examples. Team credentials supplied separately with verifiable references
5	**Cost**	§12 — effort model, five cost drivers in order of influence, six items required to finalise a fixed price. Phase structure makes effort derivable	Transparent basis. **No cost invented**
6	**Date of delivery**	§11 phase table with acceptance per phase; §11.1 critical path identifying the merge chain as the binding constraint; §12.3 the dependencies a date requires	Dependency-aware structure makes a schedule derivable on agreement of start date and staffing. **No date invented**

15 · Conclusion

This proposal offers a framework whose completeness claim can be audited. The denominator is computed rather than assumed, per configuration, with every exclusion carrying a reason a reviewer can contest individually. Privileged coverage is reported per extension and per XLEN, against a recorded baseline, with 14 of 28 named privileged extensions committed and the remaining 14 identified as future rather than quietly implied. Every scenario carries a negative control, and a control that passes is reported as a defect in our framework rather than absorbed into a pass rate.

Three positions in this document are commercially inconvenient and are stated anyway. Two of the five deliverables depend on decisions by upstream maintainers and are therefore not within our unilateral control. The one model change most likely to be rejected is identified as such, with its

fallback. And the Hypervisor extension is excluded from scope entirely rather than included at a token level.

> THE COMMITMENT
>
> 100% of the enumerated reachable denominators for each required configuration, with every unreachable target explicitly justified — measured alongside Sail branch coverage, audited by mutation sensitivity, and cross-checked against an independently written simulator.

We do not claim complete ISA coverage, because no methodology can deliver it and no honest denominator exists for the claim. We would rather state that boundary than imply we had crossed it.

APPENDICES — THE MASTER TECHNICAL PLAN

The 35-page Master Technical Plan is the authoritative engineering source and accompanies this proposal in full as Appendices A–H. This document references it rather than reproducing it.

Quantities attributed to the Sail model are properties of the upstream source obtained by static analysis of its declarations, recomputed at the revision agreed at project start.
No delivery date, cost, coverage percentage, merged contribution or completed implementation is claimed in this document. Prototype evidence appears only in §3.3, with its status stated.